

JAVA INTERVIEW PREPARATION

CHAPTER 11

RECORDS AND DATA-ORIENTED MODELING

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Records and Data-Oriented Modeling: Transparent Values with Strong Invariants

Senior / Lead Java Developer Reading Chapter

Records became a permanent Java feature in Java 16. They provide a concise way to declare classes whose primary purpose is to carry a fixed set of values. The benefit is not simply fewer lines of getters, constructors, `equals`, `hashCode`, and `toString`. A record makes a semantic claim: its state description is its API, and instances should be understood as transparent values rather than identity-rich mutable objects.

That claim is powerful, but it also creates design constraints. Record components become part of construction, access, equality, deconstruction, serialization shape, and often external contracts. Senior developers should know when this transparency is desirable, how to preserve immutability, how validation works, and why records are not automatic replacements for entities or every DTO.

The State Description Defines the Record

A traditional immutable carrier contains repeated structural code:

```
final class CustomerSummary {
    private final CustomerId id;
    private final String displayName;

    CustomerSummary(CustomerId id, String displayName) {
        this.id = id;
        this.displayName = displayName;
    }

    CustomerId id() { return id; }
    String displayName() { return displayName; }

    // equals, hashCode, and toString
}
```

A record declares the same essential state directly:

```
record CustomerSummary(CustomerId id, String displayName) {}
```

The compiler provides:

- A private final field for each component.
- A public accessor named exactly after each component.
- A canonical constructor matching the component list.
- Value-based `equals` and `hashCode` using all components.
- A `toString` describing the record name, components, and values.

```
record header
|
+--> fields
+--> canonical construction
+--> accessors
+--> equality and hashing
+--> textual representation
+--> record-pattern deconstruction
```

The header is therefore not a shortcut for private implementation details. It is the declared state model.

Record Semantics versus Ordinary Classes

Every record implicitly extends `java.lang.Record`, is final, and cannot extend another class. It may implement interfaces.

```
record Money(BigDecimal amount, Currency currency)
    implements Comparable<Money> {

    @Override
    public int compareTo(Money other) {
        requireSameCurrency(other);
        return amount.compareTo(other.amount);
    }
}
```

Records may define methods, static fields, static methods, nested types, and constructors. They may not declare additional instance fields outside their components. This keeps the complete instance state visible in the header.

An ordinary class remains appropriate when:

- Identity matters independently of current field values.
- The object has a mutable lifecycle.
- Encapsulation requires hiding the representation.
- Subclassing is part of the design.
- Construction state differs substantially from public state.
- Equality should use only a stable identifier or another custom model.

Choose a record because transparent value semantics fit the domain, not because the syntax is shorter.

Canonical and Compact Constructors

The canonical constructor has parameters matching the record components.

```
record EmailAddress(String value) {
    EmailAddress(String value) {
        this.value = normalizeAndValidate(value);
    }
}
```

A compact constructor removes the parameter list and implicit field assignments:

```
record EmailAddress(String value) {
    EmailAddress {
        Objects.requireNonNull(value, "value");
        value = value.trim().toLowerCase(Locale.ROOT);

        if (!value.contains("@")) {
            throw new IllegalArgumentException("invalid email address");
        }
    }
}
```

At the end of a compact constructor, the compiler assigns the possibly normalized parameters to their corresponding fields.

```

caller value
  |
  v
compact constructor validation/normalization
  |
  v
implicit component assignment
  |
  v
valid record instance

```

A compact constructor cannot explicitly assign `this.value`; doing so would conflict with the implicit assignment. Validate all cross-component invariants there so every normal construction path produces a valid value.

Additional Constructors Must Delegate

Additional record constructors must delegate to the canonical constructor, directly or indirectly.

```

record DateRange(LocalDate start, LocalDate end) {
    DateRange {
        Objects.requireNonNull(start);
        Objects.requireNonNull(end);
        if (end.isBefore(start)) {
            throw new IllegalArgumentException("end precedes start");
        }
    }

    DateRange(LocalDate singleDay) {
        this(singleDay, singleDay);
    }
}

```

This prevents an alternate constructor from bypassing component initialization. Static factories can provide named construction policies:

```

static DateRange month(YearMonth month) {
    return new DateRange(month.atDay(1), month.atEndOfMonth());
}

```

Factories are useful when construction needs a descriptive name, several source forms, caching, or a result that may fail without throwing.

Shallow Immutability and Defensive Copying

Record fields are final, but referenced objects may still be mutable.

```

record OrderDraft(List<OrderLine> lines) {}

List<OrderLine> source = new ArrayList<>();
OrderDraft draft = new OrderDraft(source);
source.add(new OrderLine(...)); // changes draft.lines()

```

The final field prevents replacing the List reference; it does not freeze the List.

Defensively copy at construction:

```

record OrderDraft(List<OrderLine> lines) {
    OrderDraft {
        lines = List.copyOf(lines);
    }
}

```

`List.copyOf` creates an unmodifiable snapshot and rejects null elements. It is still a shallow copy: mutable `OrderLine` objects remain shared. Prefer immutable element types or perform a justified deeper copy.

Arrays need special care because both construction and access can expose mutation:

```
record Digest(byte[] bytes) {
    Digest {
        bytes = bytes.clone();
    }

    @Override
    public byte[] bytes() {
        return bytes.clone();
    }
}
```

Without the accessor copy, callers can mutate the record's internal array after construction.

Defensive copying does not change array equality. The generated record `equals` uses the array's own `equals`, which is reference equality, not content equality. Two `Digest` instances containing separate arrays with identical bytes therefore compare unequal. If content equality is required, use a component type with content-based equality or choose an ordinary class with carefully implemented `equals` and `hashCode`; overriding record equality is possible but weakens the simplicity of generated transparent semantics.

Equality and Numeric Value Traps

Record equality includes every component and uses the component values' `equals` behavior.

```
record Price(BigDecimal amount) {}

new Price(new BigDecimal("10.0"))
    .equals(new Price(new BigDecimal("10.00"))); // false
```

`BigDecimal.equals` considers scale. If the domain considers these monetary values equal, normalize scale in the compact constructor or use a `Money` type with deliberate equality semantics.

Do not override record equality casually. The record's transparent state, accessors, deconstruction, and equality should describe the same value model. If equality ignores components or relies on hidden policy, an ordinary class may communicate the design more honestly.

Mutable components also make records dangerous as Map keys. If a component's hash changes after insertion, lookup can fail just as it can for any mutable key.

Accessors Are Not JavaBean Getters

For component `displayName`, the generated accessor is `displayName()`, not `getDisplayName()`.

```
record CustomerView(String displayName, boolean active) {}

view.displayName();
view.active();
```

Modern frameworks generally support records, but older reflection libraries, template engines, mapping tools, or bean-based conventions may expect `getX` and a no-argument constructor. Verify actual framework and version support rather than assuming every DTO mechanism understands record metadata.

You can override an accessor, but it must return the component type:

```
record AccountNumber(String value) {
    @Override
    public String value() {
        return value;
    }
}
```

Changing the meaning on access, such as returning a freshly normalized value different from equality state, makes transparent semantics harder to reason about. Normalize during construction instead.

Records as DTOs and API Models

Records work well for request, response, query, event, and projection data when the component list is genuinely the public shape.

```
record CreateOrderRequest(
    CustomerId customerId,
    List<CreateOrderLineRequest> lines
) {
    CreateOrderRequest {
        Objects.requireNonNull(customerId);
        lines = List.copyOf(lines);
        if (lines.isEmpty()) {
            throw new IllegalArgumentException("order requires at least one line");
        }
    }
}
```

However, changing a record component can affect constructor signatures, JSON, schema generation, mapping, equality, tests, and record patterns. A public record header is an exposed contract.

Do not use the same record automatically for database projection, internal command, domain value, and external API response. Those layers evolve for different reasons. Separate models reduce accidental coupling even when they initially contain similar fields.

JSON and Serialization Boundaries

Record support varies by serializer version and configuration. A serializer typically uses record components as properties and the canonical constructor for deserialization.

Validation in the compact constructor is valuable because deserialization cannot create an invalid instance through that constructor. But error translation still matters: a constructor exception should become a useful client error rather than a generic server failure.

Java serialization treats records specially through their canonical construction model, but Java native serialization is rarely a good long-term wire or persistence format. Prefer explicit schemas and stable protocols for events and remote APIs.

Be careful with sensitive data. Generated `toString` includes component values:

```
record LoginCommand(String username, char[] password) {}
```

Logging this record can expose secrets, and the mutable array breaks value safety. Model secrets with dedicated protected types, avoid including them in general-purpose record state, or override textual representation carefully.

Records and JPA Entities

Records are generally poor JPA entity types. JPA entities often require provider-managed construction, proxies, mutable state, lazy associations, and identity semantics. Records are final, have fixed final components, and describe value equality across all components.

Records are useful as query projections:

```
record CustomerOrderTotal(  
    CustomerId customerId,  
    BigDecimal total  
) {}
```

A query can populate a projection without turning it into a managed entity. Keep transaction and lazy-loading boundaries explicit before creating the record. Passing a proxied association into a record does not initialize it safely or detach it automatically.

Entity identity and value-object equality answer different questions:

JPA entity	record/value
-----	-----
identity across time	equality from components
lifecycle mutation	fixed component references
provider management	transparent construction
lazy proxies possible	final class

Records Are Not Builders

Records require all components at construction. A builder can be appropriate when input is gathered in stages, many optional values exist, or validation needs a readable construction DSL.

```
OrderCommand command = OrderCommandBuilder.anOrder()  
    .forCustomer(customerId)  
    .withLine(productId, quantity)  
    .requestedBy(userId)  
    .build();
```

The builder may produce a validated record as its final immutable value. Do not add many nullable record components merely to avoid a builder. A large header with unrelated optional values signals that the model may contain several distinct cases.

Records in Domain Modeling

Records are excellent for small value objects:

```
record CustomerId(UUID value) {  
    CustomerId {  
        Objects.requireNonNull(value);  
    }  
}  
  
record Quantity(int value) {  
    Quantity {  
        if (value <= 0) {  
            throw new IllegalArgumentException("quantity must be positive");  
        }  
    }  
}
```

These types replace primitive obsession and make invalid combinations harder to express.

Records can also carry behavior that belongs to the value:

```
record Money(BigDecimal amount, Currency currency) {
    Money add(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }
}
```

Data-oriented does not mean behavior-free. The key is that behavior operates naturally on the transparent value rather than managing a hidden mutable lifecycle.

Local and Nested Records

Records can be nested or local to a method. A local record gives a complex intermediate result a name without expanding the public API.

```
List<CustomerTotal> calculateTotals(List<Order> orders) {
    record Accumulator(CustomerId customerId, BigDecimal total) {}

    return orders.stream()
        .map(order -> new Accumulator(order.customerId(), order.total()))
        .collect(groupingAndSumming())
        .stream()
        .map(value -> new CustomerTotal(value.customerId(), value.total()))
        .toList();
}
```

Use local records when the named structure improves an algorithm. Avoid creating one for every pair of temporary variables when a small existing domain type is clearer.

Records with Sealed Types and Patterns

Records combine naturally with sealed hierarchies because each permitted case can expose a fixed payload.

```
sealed interface PaymentResult
    permits PaymentResult.Approved, PaymentResult.Declined {

    record Approved(TransactionId id) implements PaymentResult {}
    record Declined(DeclineReason reason) implements PaymentResult {}
}
```

```
PaymentResult
|
+-- Approved(transactionId)
+-- Declined(reason)

closed alternatives + transparent case data
```

Java 21 record patterns can deconstruct these values in pattern matching. This supports data-oriented processing where the compiler can check that all permitted alternatives are handled.

Versioning and Binary Compatibility

Changing a record header is a significant API change. Adding, removing, reordering, or changing component types changes the canonical constructor and may break compiled callers. It can also alter equality, hashing, `toString`, serialization, JSON, and patterns.

For public libraries and long-lived events, do not assume that concise syntax makes evolution inexpensive. Use versioned DTOs, tolerant readers, schema compatibility rules, or an ordinary class with a more stable

abstraction when representation must evolve independently.

Within a service, records are easier to evolve when their scope is narrow and consumers compile together. Still search for constructor calls, mappings, persistence projections, and serialized uses before changing components.

Testing Records

Test domain semantics rather than compiler-generated boilerplate.

```
@Test
void normalizesEmailDuringConstruction() {
    EmailAddress email = new EmailAddress(" USER@Example.COM ");

    assertEquals("user@example.com", email.value());
}
```

Important tests include:

- Null rejection and cross-component validation.
- Defensive copies for collections and arrays.
- Normalization and equality boundaries.
- JSON and schema compatibility.
- Secret-safe logging behavior.
- Framework construction and mapping.
- Mutation attempts through returned component objects.

There is little value in tests that merely confirm a compiler-generated accessor returns its component, unless a framework integration depends on it.

Production Perspective

Record problems usually arise when concise syntax hides a broad contract. A List component is not copied and changes after insertion into a cache key. A public component is renamed and breaks JSON consumers. Generated `toString` leaks credentials. A BigDecimal component produces surprising inequality because scale differs. A JPA entity is converted to a record and loses provider behavior. A projection captures a lazy proxy and fails after the transaction closes.

When reviewing or diagnosing a record, ask:

- Is this genuinely a transparent value or does identity matter?
- Are all components immutable or defensively copied?
- Does generated equality match the domain?
- Which wire, database, reflection, or framework contracts use the header?
- Can constructor validation explain failures at the correct boundary?
- Does `toString` expose secrets or excessive data?
- How will component changes affect binary and schema compatibility?
- Would a sealed result, builder, or ordinary class express the lifecycle better?

Interview-Ready Summary

A record declares a transparent fixed state description. The compiler provides private final component fields, accessors, a canonical constructor, value equality, hashing, and textual representation. Records are final, extend `java.lang.Record`, cannot declare extra instance fields, and may implement interfaces.

Compact constructors validate and normalize parameters before implicit field assignment. Additional constructors must delegate to canonical construction. Component references are final, but referenced objects may be mutable, so Lists, Sets, Maps, arrays, dates, and custom mutable objects require deliberate defensive-copy policy.

Record equality includes every component and inherits each component's equality semantics. This creates traps with mutable values and types such as `BigDecimal`. The header is part of the API: changing it affects construction, equality, patterns, serialization, mapping, and compatibility.

Records are strong choices for value objects, commands, query projections, events, and DTOs when their component list is the desired public shape. They are generally poor JPA entities because entity identity, proxies, lifecycle mutation, and provider construction conflict with record semantics.

Records can contain behavior and enforce rich invariants; data-oriented does not mean anemic. Use an ordinary class when identity, hidden representation, mutable lifecycle, inheritance, or independent API evolution matters more than transparent value semantics.

Revision Notes

- Records became permanent in Java 16.
- A record header declares its complete instance state model.
- The compiler generates component fields, accessors, canonical construction, equality, hashing, and `toString`.
- Records are final, implicitly extend `java.lang.Record`, cannot extend another class, and cannot declare extra instance fields.
- Records may implement interfaces and define methods, static members, and nested types.
- Compact constructors validate and normalize before implicit assignment; additional constructors must delegate to canonical construction.
- Static factories provide named construction policies.
- Final component references do not make referenced objects immutable.
- Defensively copy mutable collections and arrays at construction; array accessors may also need to return a copy.
- Array components still use reference equality unless equality is designed explicitly.
- `List.copyOf` is shallow and rejects null elements.
- Record equality includes every component; `BigDecimal` equality includes scale.
- Mutable components make records unsafe hash keys.
- Generated accessors use `componentName()`, not `JavaBean getComponentName()`.
- Verify framework, serializer, mapper, reflection, and `JavaBean` compatibility.
- Normalize during construction instead of changing accessor meaning.
- Public record headers are compatibility contracts; separate models that evolve for different reasons.
- Generated `toString` can leak secrets.
- Records are poor JPA entities but useful query projections.
- Builders suit staged construction; many nullable components may indicate distinct domain cases.
- Records suit validated values and local intermediates and may contain behavior.
- Records combine naturally with sealed hierarchies and record patterns.
- Component changes affect construction, equality, serialization, patterns, and compatibility; test these boundaries and choose records for transparent values, not merely less boilerplate.